

Program execution

From source code to program execution

Introduction

- We'll take a break from Assembler and machine instructions to see what's involved in getting a source program to run
- We'll see that there are three steps:
 1. Assembly (what high-level languages call compiling)
 2. Linking
 3. Execution

Program assembly

- Let's look at the Assembler listing of our demo program:

Location	Object Code	Source Statement
000000		DEM01 CSECT
000000	C0C0 0000 0000	LARL 12,DEM01 Load base reg.
		USING DEM01,12 Use R12 as base
000006	4830 C00E	LH 3,Int1 Load Int1 into R3
00000A	18F3	LR 15,3 Set return code
00000C	07FE	BR 14 End program
00000E	000A	Int1 DC H'10'
		END

Program assembly

- On the right, we have our **source program**:

Location	Object Code	Source Statement	
000000		DEM01	CSECT
000000	C0C0 0000 0000		LARL 12,DEM01 Load base reg.
			USING DEM01,12 Use R12 as base
000006	4830 C00E		LH 3,Int1 Load Int1 into R3
00000A	18F3		LR 15,3 Set return code
00000C	07FE		BR 14 End program
00000E	000A	Int1	DC H'10'
			END

Program assembly

- On the left, the object code:

Location	Object Code	Source Statement	
000000		DEM01	CSECT
000000	C0C0 0000 0000		LARL 12,DEM01 Load base reg.
			USING DEM01,12 Use R12 as base
000006	4830 C00E		LH 3,Int1 Load Int1 into R3
00000A	18F3		LR 15,3 Set return code
00000C	07FE		BR 14 End program
00000E	000A	Int1	DC H'10'
			END

Program assembly

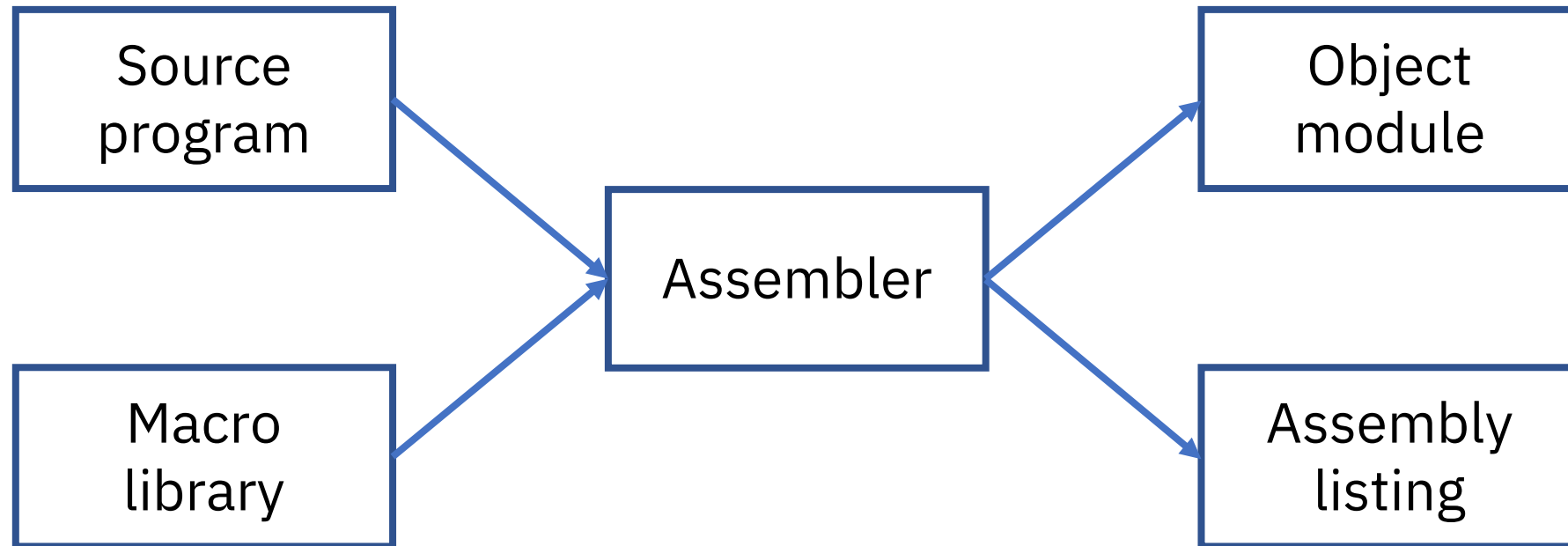
- Program assembly is the process of converting **source code** to **object code**

Location	Object Code	Source Statement
000000		DEM01 CSECT
000000	C0C0 0000 0000	LARL 12,DEM01 Load base reg.
		USING DEM01,12 Use R12 as base
000006	4830 C00E	LH 3,Int1 Load Int1 into R3
00000A	18F3	LR 15,3 Set return code
00000C	07FE	BR 14 End program
00000E	000A	Int1 DC H'10'
		END



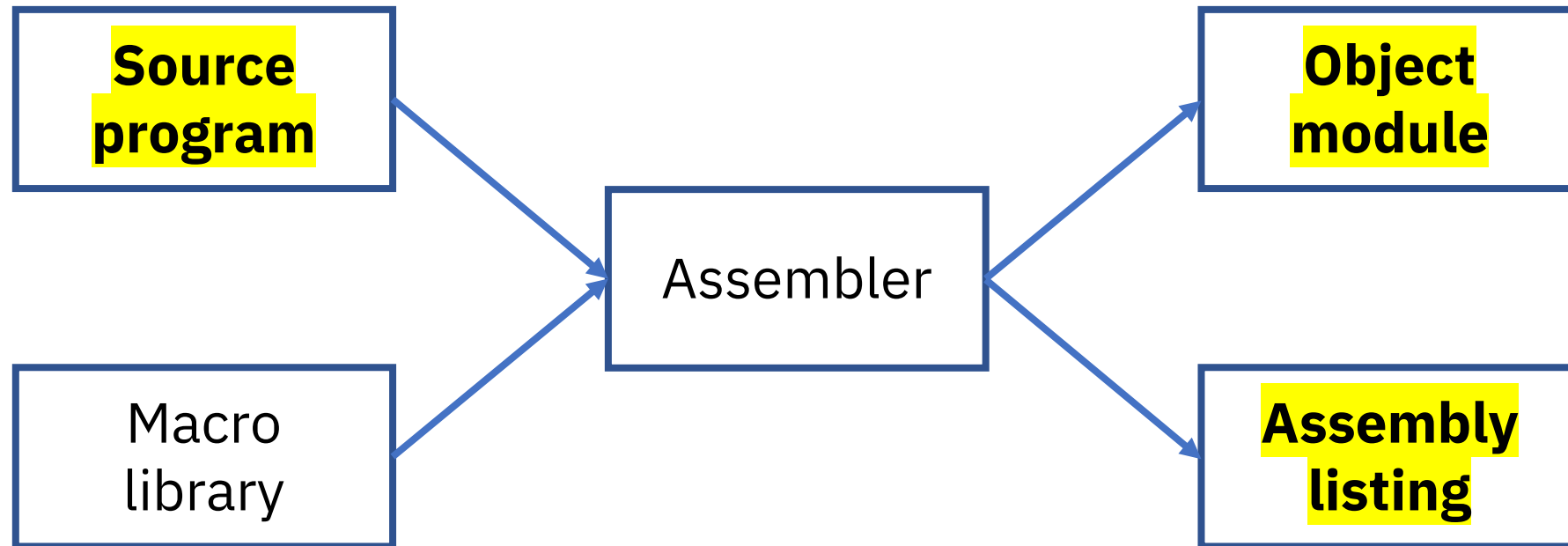
Program assembly

- In z/OS, the process looks like this:



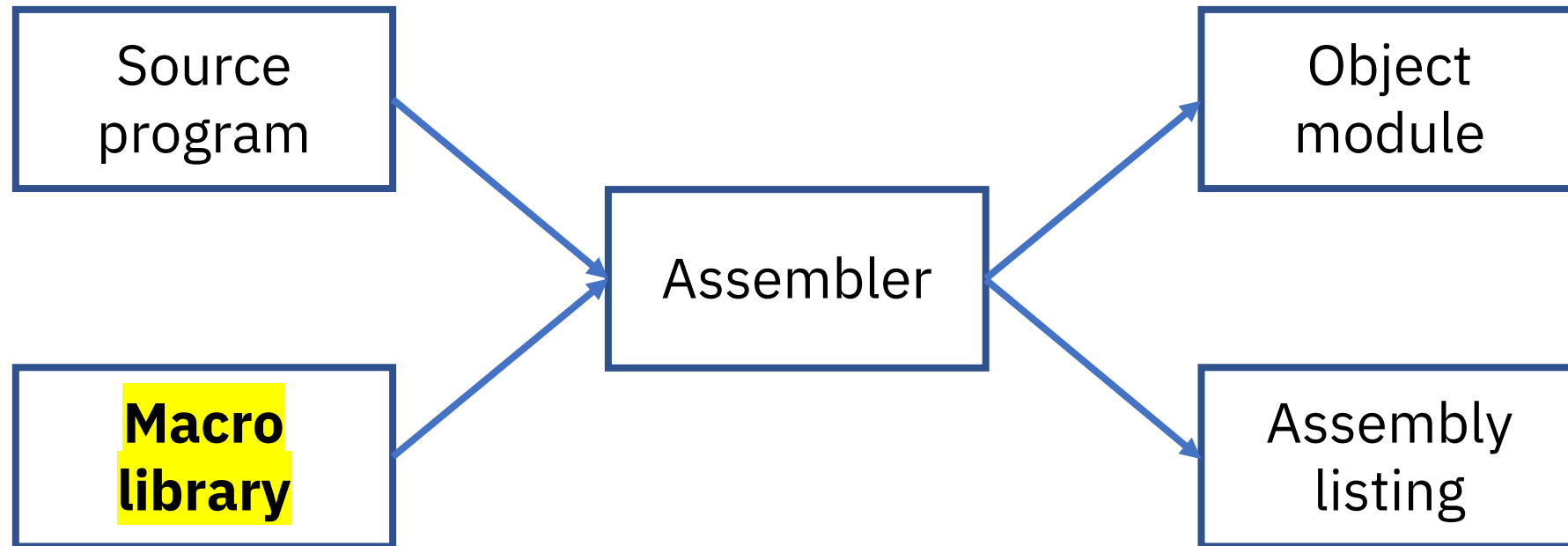
Assembler files

- The Assembler converts your **source** program to **object (machine) code** and produces a **listing**



Program assembly

- The **macro library** is described next

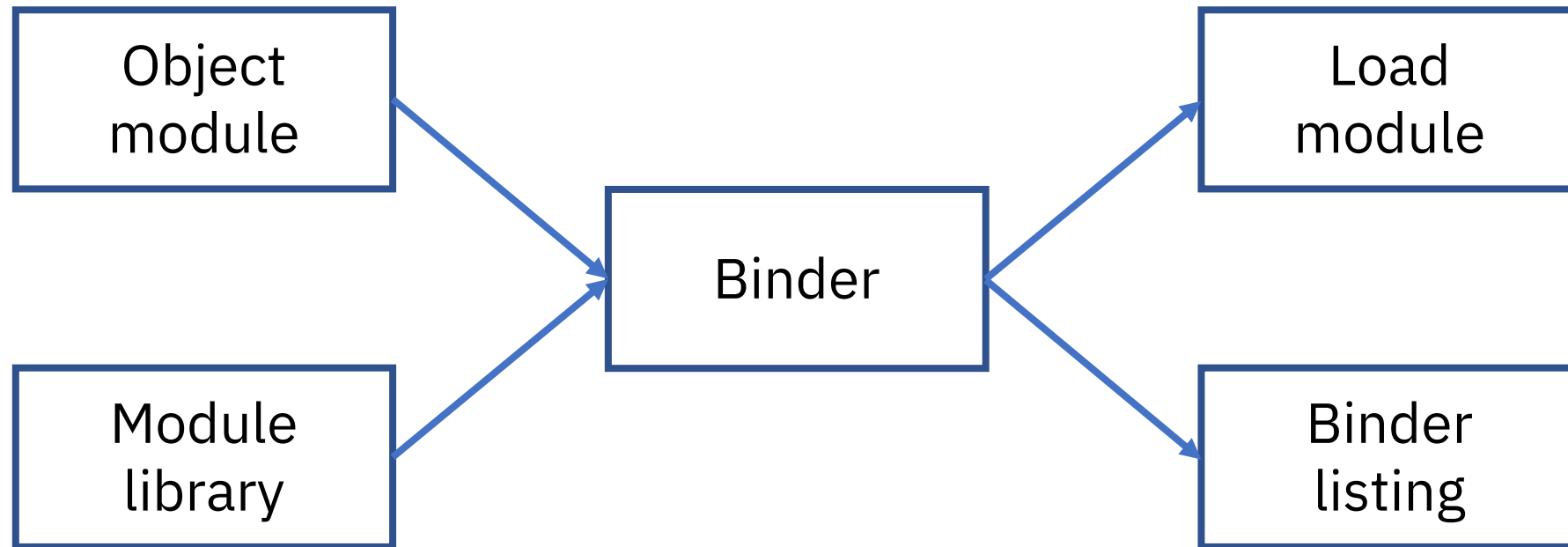


Macro library

- Assembler **macros** provide a compact notation for groups of statements
 - You write one statement (the macro) and the Assembler generates a group of statements
- They are used to interact with the operating system, for example to read from and write to files
- You can also reuse data definitions (like the Employee example) by:
 - putting the data structure definition in a macro library and
 - copying it from there into your programs

Linking

- The object module is not an executable program yet. The **Binder** (also known as **Linkage Editor**) processes the object module to produce an *executable load module*
- Like the Assembler, it also produces a listing

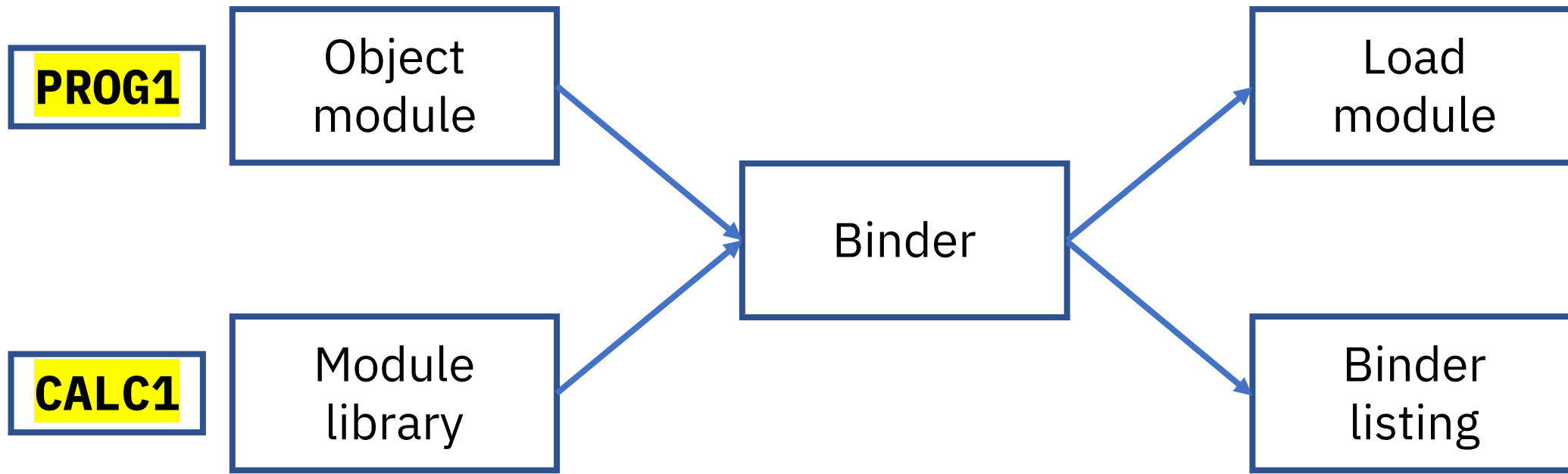


The module library

- The module library contains object modules and load modules
- Your executable load module may consist of more than just your program
 - Your program may call other programs or subroutines which are then needed in the executable module
- The Binder combines your object module with any others needed for execution to produce the load module

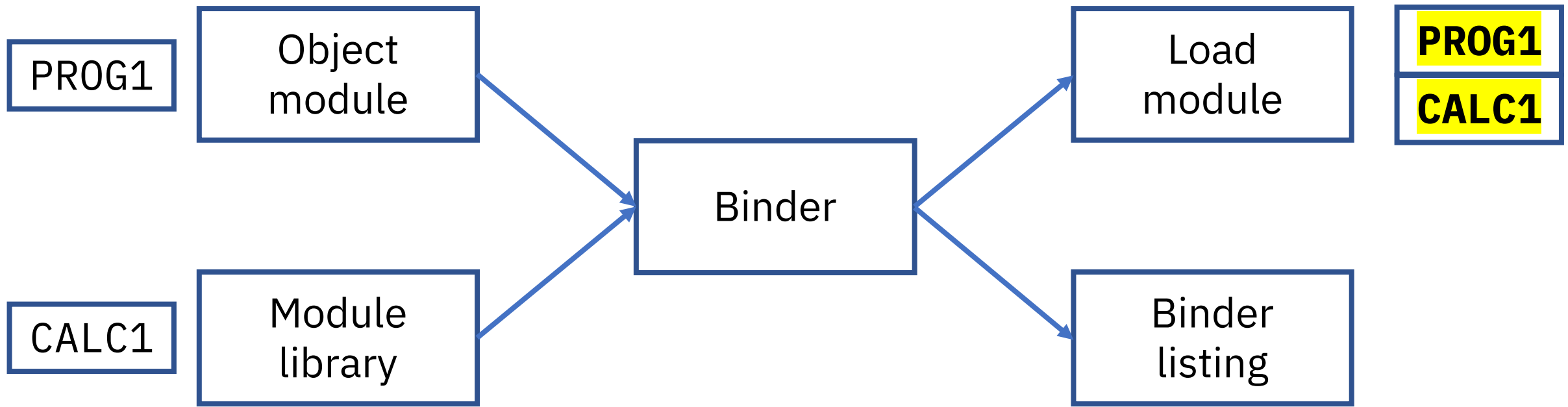
The module library - example

- Your program, **PROG1**, calls another program called **CALC1**
- The object module for **CALC1** is in the module library



The module library - example

- The Binder combines both modules in the executable load module:



The load module - execution

- The load module is usually written as a member of a partitioned dataset (PDSE)
- For example, the Binder could write our demo program as member **DEM01** of the PDSE **MY.LOADLIB**:

MY.LOADLIB(DEM01)

- These JCL statements would then run the program:

```
//RUNDEMO EXEC PGM=DEM01  
//STEPLIB DD DSN=MY.LOADLIB, DISP=SHR
```

Stored procedures

- z/OS provides JCL stored procedures to assemble, link and run programs
- They are:
 - **ASMAC** to assemble a program (the “C” is for “compile”)
 - **ASMACL** to assemble and link a program (“L” is for “link”)
 - Use it to place the module in a PDSE for linking or running later
 - **ASMACLG** to assemble, link, and run a program (the “G” is for “go”)
 - Use it to assemble, link, and run a program, in a single job
- Next, how to use ASMACLG with our DEMO1 program

Running our DEMO1 program

- This JCL assembles, links, and runs our DEMO1 program:

//DEM01 JOB ...			
//STEP1 EXEC PROC=ASMACLG			
//C.SYSIN DD *			
DEM01	CSECT		
	LARL	12,DEM01	Load base register
	USING	DEM01,12	Use R12 as base
	LH	3,Int1	Load Int1 into R3
	LR	15,3	Set return code
	BR	14	End program
Int1	DC	H'10'	
	END		

Run the ASMACLG stored procedure

Running our DEMO1 program

- This JCL assembles, links, and runs our DEMO1 program:

//DEM01 JOB ...		Run the ASMCLG stored procedure
//STEP1 EXEC PROC=ASMACLG		
//C.SYSIN DD *		Input to the “C” (compile) step
DEM01	CSECT	
	LARL 12,DEM01 Load base register	
	USING DEM01,12 Use R12 as base	
	LH 3,Int1 Load Int1 into R3	
	LR 15,3 Set return code	
	BR 14 End program	
Int1	DC H'10'	
	END	

Running our DEMO1 program

- This JCL assembles, links, and runs our DEMO1 program:

//DEM01 JOB ...		Run the ASMCLG stored procedure
//STEP1 EXEC PROC=ASMCLG		
//C.SYSIN DD *		Input to the “C” (compile) step
DEM01	CSECT	
	LARL 12,DEM01	Load base register
	USING DEM01,12	Use R12 as base
	LH 3,Int1	Load Int1 into R3
	LR 15,3	Set return code
	BR 14	End program
Int1	DC H'10'	
	END	

Running our DEMO1 program

- When the job ends, we'll see these messages in the job log:

13.26.29	JOB34544	-STEPNAME	PROCSTEP	RC
13.26.29	JOB34544	-STEP1	C	00
13.26.29	JOB34544	-STEP1	L	00
13.26.29	JOB34544	-STEP1	G	10
13.26.30	JOB34544	-DEMO1	ENDED.	

- Note the return codes for each step

Conclusion

- This ends our brief introduction to the process needed to run our source program
- We have reached the end of this course! Hope you liked it
- In the next course, we'll see machine instructions in detail, and Assembler instructions and z/Architecture concepts we haven't yet covered